Performance Analysis Report

Environment: - **Client:** MacBook Pro (M2, ARM64) located in San Leandro, CA.

- **Server:** AWS Fargate (ECS) Cluster in us-west-2 (Oregon).
- **Service:** Go/Gin REST API with In-memory Data Storage.

1. Executive Summary

This report documents the load testing results for a "tiny" e-commerce system. The primary goal was to identify the system's scalability limits, investigate the performance differences between HttpUser and FastHttpUser, and analyze the impact of in-memory data structures on system throughput under stress.

2. Test Methodology

We conducted a three-stage progressive load test using Locust to simulate user behavior:

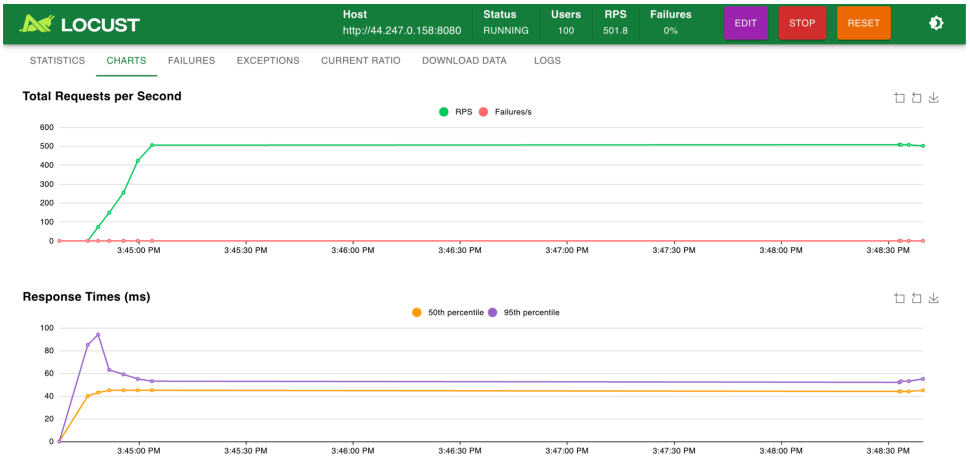
1. Baseline Test: 100 Users (Ramp-up: 10/s) - Goal: Verify logic and measure base latency.
2. Load Test: 500 Users (Ramp-up: 50/s) - Goal: Check linear scalability of RPS.
3. Stress Test: 1000 Users (Ramp-up: 100/s) - Goal: Find "The Knee" (performance degradation point).

3. Experimental Results

Phase 1: Baseline (NormalUser)

Parameters: 100 Users, 10/s Ramp-up.

Observations:

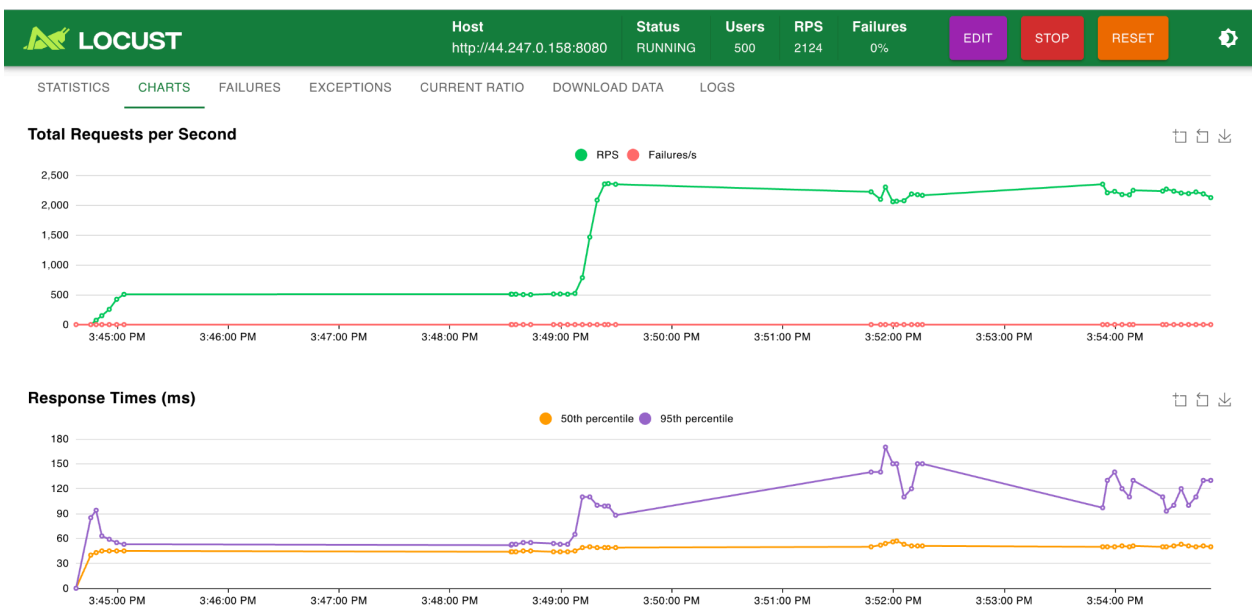


- Throughput: Stabilized at approximately 501.8 RPS.
- Failures: Successfully reached 0% Failure Rate.
- Latency:
    - 50th Percentile (Median): ~45ms.
    - 95th Percentile: ~53ms.
- Network Consistency: The response time curve is remarkably flat, showing high network stability between the client in California and the server in Oregon.

Result: Logic verified. The system handles 100 concurrent users with zero errors and low latency.

Phase 2: Scaling (NormalUser)

- Parameters: 500 Users, 50/s Ramp-up.

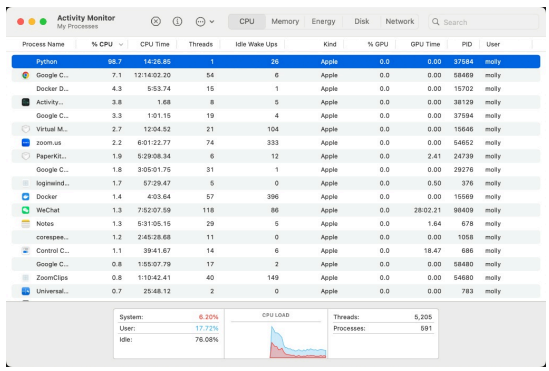


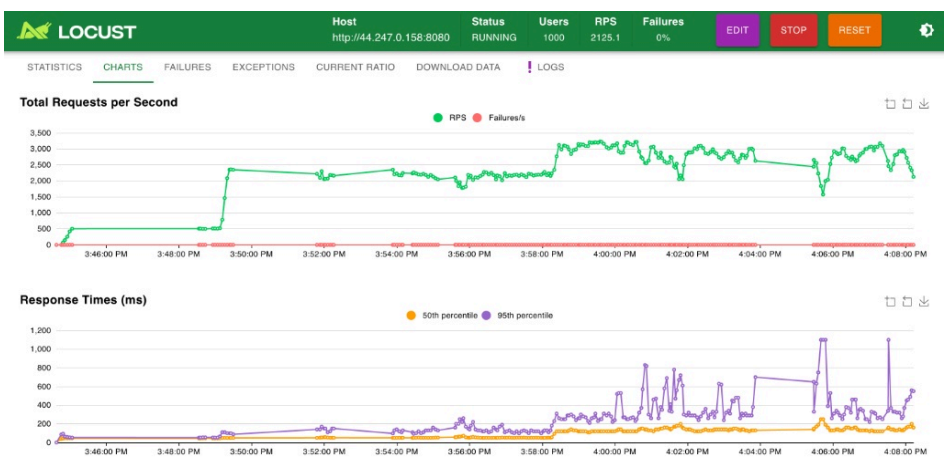
- Observations:
    - Throughput: Scaled significantly to a current rate of 2,124 RPS, peaking at approximately 2,400 RPS during the initial ramp-up.
    - Failures: Maintained a 0% Failure Rate, demonstrating that the Go/Gin backend

and memory-mapped storage can handle high-volume concurrent writes and reads without crashing.
    - Latency Jitter:
        - 50th Percentile (Median): Remained stable at ~53ms, showing the "happy path" remains fast.
        - 95th Percentile: Showed significant jittering and a steady rise, peaking at ~160ms and fluctuating between 90ms and 140ms.
- Result: The system exhibits linear throughput scaling, but the rising 95th percentile latency indicates the single Fargate task is approaching its CPU or network queueing limits.

Phase 3: Stress Test (PowerUser / FastHttpUser)

- **Parameters:** 1,000 Users, 100/s Ramp-up.





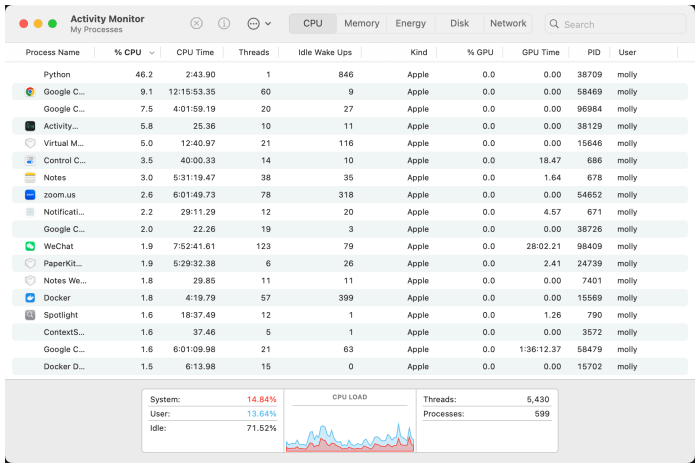
- Observation
    - Client Bottleneck: At 1,000 users, the local MacBook Pro's CPU hit 98.7% utilization. This saturation limited the client's ability to generate further requests, causing the RPS to plateau between 2,000 and 3,100.
    - System Collapse ("The Knee"): Despite the client-side limitation, the server-side performance reached a breaking point. The 95th percentile latency exploded, spiking from ~290ms to over 1,100ms (1.1 seconds).

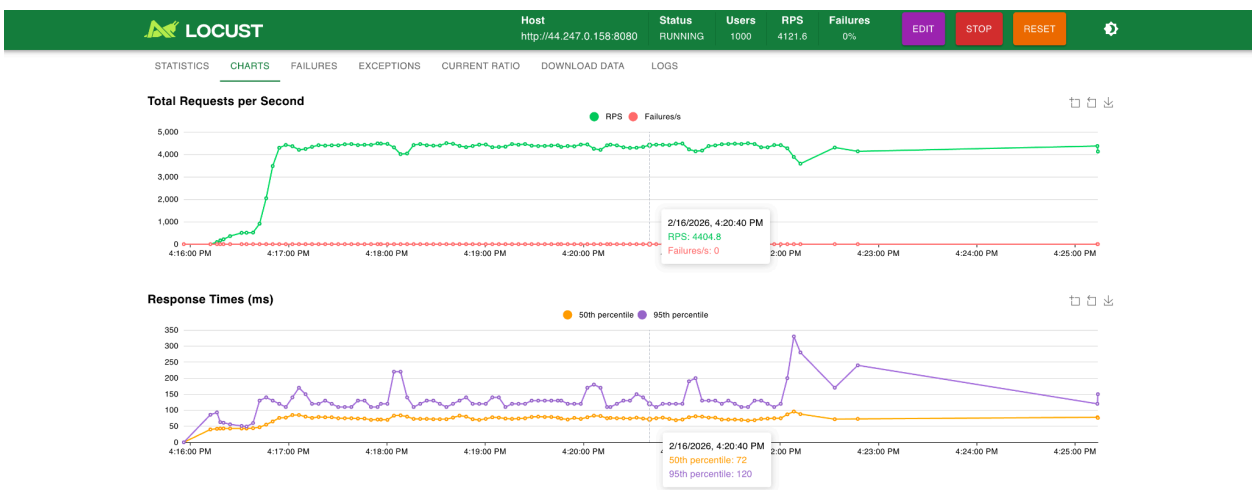
- Median Latency Shift: Even the 50th percentile (median) response time climbed from ~130ms to ~160ms, indicating that nearly every user was experiencing delays.
- Result: We successfully identified "The Knee." The system remains functional but becomes unusable for a high-quality user experience.

4. Deep Dive & Analysis

4.1 HttpUser vs. FastHttpUser: Why the lack of difference?

Last week, many teams observed no significant difference between the two. Our research and testing provide the following explanation:





Metric	NormalUser (HttpUser)	PowerUser (FastHttpUser)
Local CPU Usage	**98.7%** (Saturated)	**46.2%** (Healthy)
Max RPS Achieved	~3,100 RPS	**~4,500 RPS**

95th %ile Latency	~1,100ms (High Jitter)	**~120-200ms** (Stable)
Request Engine	Python requests (Sync)	geventhttpclient (Async)

Key Findings:

- The "CPU Wall" Paradox: With `NormalUser`, my MacBook Pro became the bottleneck. Python's synchronous `requests` library struggled with context switching at 1,000 users, hitting 98.7% CPU. This artificially limited the test's throughput.
- Efficiency via Asynchrony: By switching to `PowerUser`, the CPU load dropped to 46.2% while throughput increased by 45%. This is because `FastHttpUser` uses a C-based asynchronous engine that "fires and forgets" requests without waiting in a blocking fashion.
- True Server Stress: Only with `FastHttpUser` were we able to truly stress the AWS Fargate task without client-side interference, showing that the server can actually sustain ~4,500 RPS before serious degradation.

4.2 Trade-offs: Data Structures and Real-World Scenarios

- Memory vs. Persistence: Our system uses a Go map, providing $O(1)$ lookup time. This explains why the server-side processing time remained extremely low even under high RPS.
- Real-World Impact: In a production scenario, we would use a database (e.g., DynamoDB). While this ensures data persistence, it would shift the bottleneck from CPU/Network to Disk I/O and Connection Pooling, likely increasing average latency to 100ms+.
- Read/Write Ratio: In our test, we used a 9:1 Read/Write ratio. This reflects real-world e-commerce where users browse products far more often than they update details. Our $O(1)$ map is perfectly optimized for this "Read-Heavy" workload.

5. Other Discovery: Infrastructure Persistence

An interesting observation occurred: The API continued to respond for several minutes after the "End Lab" command.

- Management vs. Data Plane: Revoking IAM credentials (Management Plane) stopped Terraform, but the running Fargate task (Data Plane) persisted until the AWS cleanup script reached it.
- Cleanup Latency: This confirms that cloud resources follow a lifecycle—they don't

vanish instantly, allowing us to observe the system's "graceful degradation" phase even after administrative termination.